Linnæus University
Sweden

Lab Assignment 1

LEGv8 Assembly Programming

Computer Technology 1

October 1, 2026



Author: Andres Rodriguez
Examiner: Thomas
Semester: HT25
Discipline: Exchange Erasmus Student
Course code: 1dt301

Task 1 – Basic Instructions in LEGv8

Write the following code in the simulator and run it: (Assemble and execute all instructions)

Listing 1: LEGv8 program for Task 1

```
MOVZ X0, #5
MOVZ X1, #10
ADDI X1, X1, #2
ADD X2, X0, X1
```

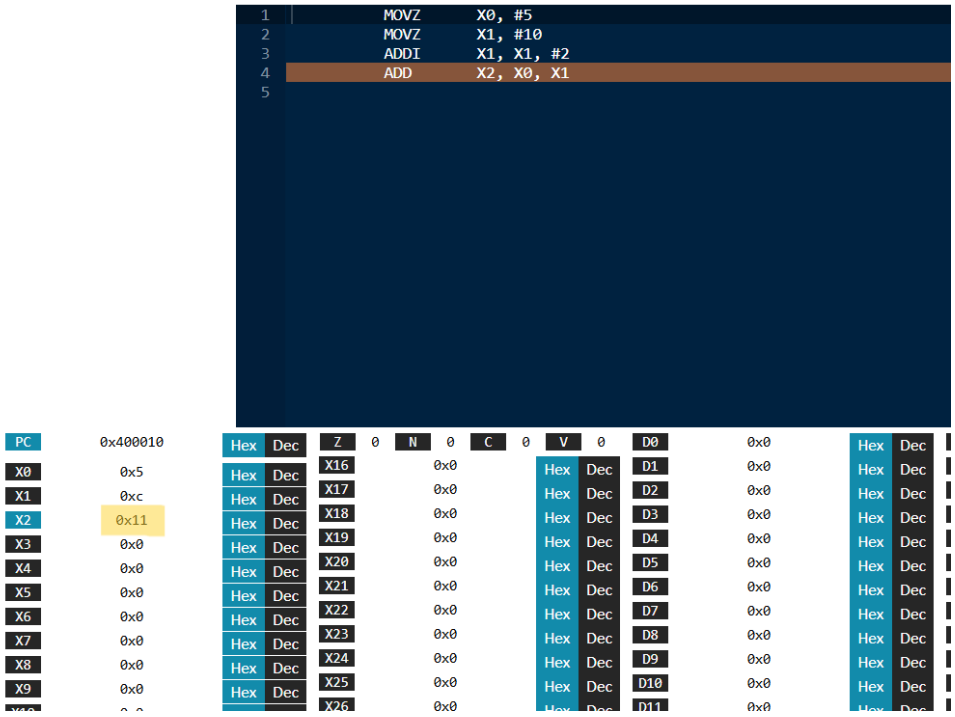


Figure 1: Execution of Task 1 in the LEGv8 simulator showing final values in registers.

Answer

The number stored in register X2 is 17 (deminal) or 11 (hex)

Task 2 – Machine Code to LEGv8 Assembly

Below is an example, Remove this and add the given instruction binaries.

Instruction 1

Machine Code: `11010010100000000001000000000010`

Assembly:

```
MOVZ x2,#128,LSL 0
```

Explanation: Move intenger 128 to register x2, shifting 0 positions . Instruction 2

Machine Code: `11010010100000000001110011100100`

Assembly:

```
MOVZ x4,#231,LSL 0
```

Explanation: Move integer 231 to register x4, shifting 0 positions . Instruction 3

textbfInstruction 3 **Machine Code:** `11001011000000100000000010000101`

Assembly:

```
SUB x5,x4,x2
```

Explanation: Subtract register x4 minus x2 and store the result in register x5 . Instruction 4

Machine Code: `D360 0CA5`

Assembly:

```
LSL x5,x5,#6
```

Explanation: Takes register x5 and shift 6 positions to the left, storing number in register x5 .

Task 3 - LEGv8 Arithmetic

Create a LEGv8 Assembly program to calculate the value of the following expression:

$$4 \cdot 5 + 16 \cdot 11 + 25$$

When finished, the result shall be stored in register x0.

Code

Listing 2: LEGv8 program for Task 3

```
MOVZ X2, #5
LSL X2, x2, 2
MOVZ X1, #11
LSL X1, x1, 4
ADD x0, x2, x1
ADDI x0, x0, #25
```

Screenshot

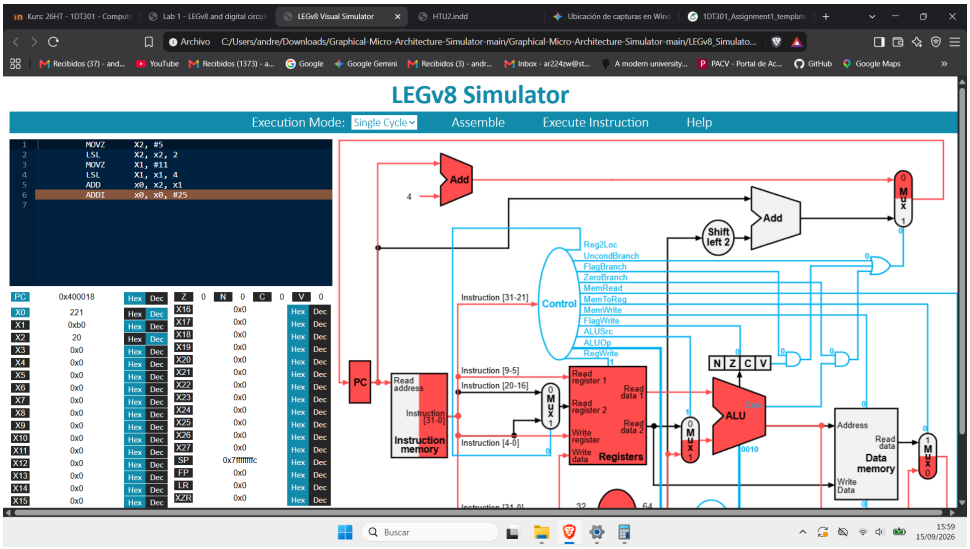


Figure 2: Execution of Task 3 in the LEGv8 simulator showing final values in registers.

Explanation

As $4 = 2^2$ and $16 = 2^4$, we shift logically 5 (101) 2 position to the left, then 11 (1011) 4 positions to the left and sum those multiplications plus 25.

Comment

As our assembler language does not have a multiply instruction if we want to multiply to other values that are not powers of two we have to create a loop than sums the number x times.For example, 3*5=3+3+3+3+3, in c++ would be: result=0; for(int i=0;i<5;i++) result+=3

Task 4 – Sum of Large Numbers

Instruction: Write a LEGv8 Assembly program to calculate the sum

$$1\,893\,423 + 443\,924$$

The numbers are decimal integers. You will probably encounter a problem to load these large numbers into registers, so you will have to find a way to solve this problem!

Code

Listing 3: LEGv8 program for Task 4

```
MOVZ x0, #0xE42F
MOVK x0, #0x001C, LSL 16
MOVZ x1, #0xC614
MOVK x1, #0x0006, LSL 16
ADD x2, x0, x1
```

Screenshot

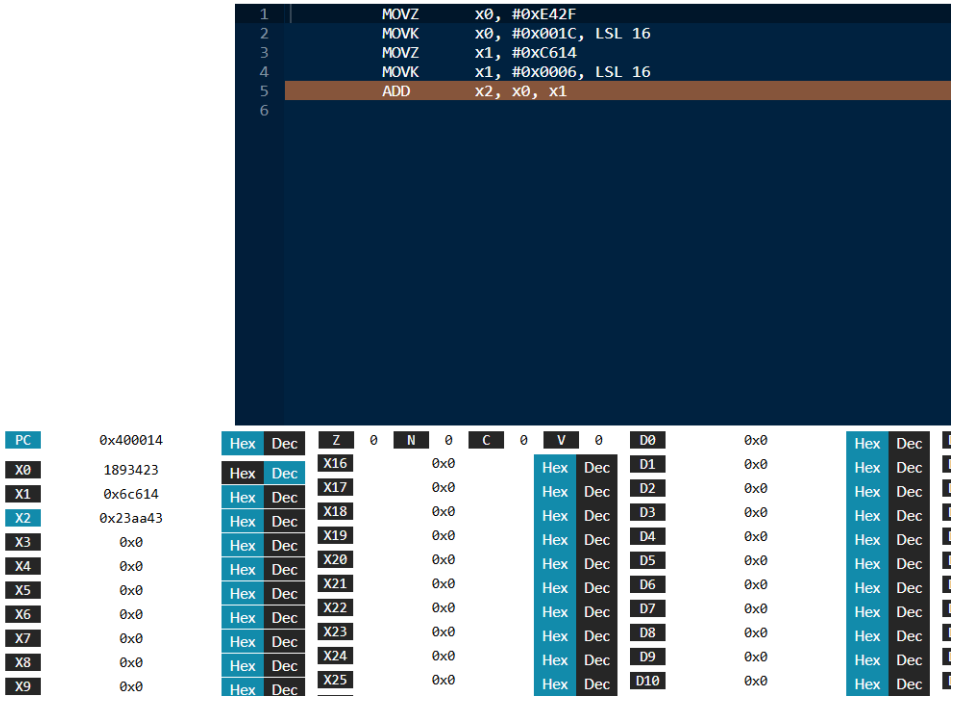


Figure 3: Execution of Task 4 in the LEGv8 simulator showing final values in registers.

Explanation

Explanation

Firstly, we have to know the hexadecimal equivalent of our decimal numbers, $1\,893\,423_{10} =$ 0x1C E42F$_{16}$ and $443\,924_{10} =$ 0x06 C614$_{16}$. The next part is just moving the lower part (`0xE42F`) with `MOVZ` and then moving the upper part (`0x001C`) shifting 16 bits (`LSL #16`) while keeping what we had—that is why we use `MOVK`. Nextly, we just do the same with the other number and sum them together using `ADD`, obtaining 0x001CE42F + 0x0006C614 = 0x0023AA43 ($2\,337\,347_{10}$).

Task 5 – Sum of Odd Numbers

Write a LEGv8 Assembly program to calculate the sum

1 + 3 + 5 + ... + 99.

When finished, the sum shall be stored in register x1.

Code

Listing 4: LEGv8 program for Task 5

```
      MOVZ x0, #1
      MOVZ x1, #0
      MOVZ x2, #99
loop:
      ADD x1, x1, x0
      MOVZ x10, #0
      SUB x10, x2, x0
      CBZ x10, final

      ADDI x0, x0, #2
      B loop
final:
```

Screenshot

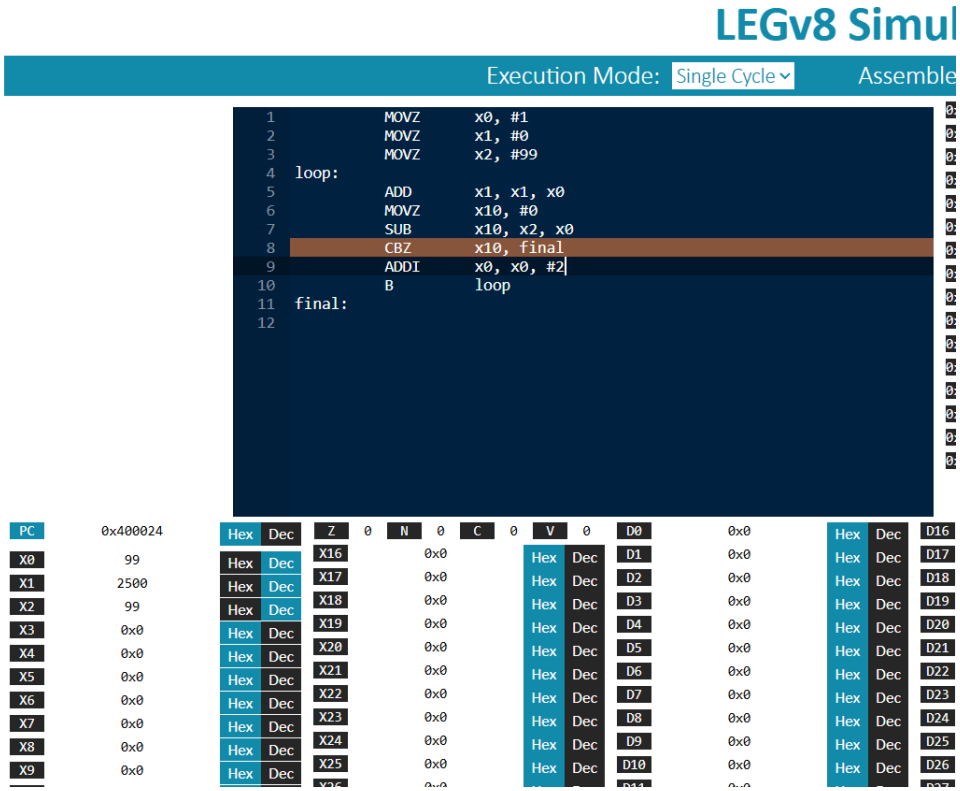


Figure 4: Execution of Task 5 in the LEGv8 simulator showing final values in registers.

Explanation

I use register `X1` from the beginning as the accumulator for the result, and register `X0` as the loop iterator initialized to `1`. In every iteration, I add the current odd number in `X0` to the previous partial result in `X1`. Then, I check if the iterator has reached 99 by subtracting `X0` from `X2` into a temporary register (`X10`) and using the `CBZ` instruction to break the loop when the difference is zero. If the limit has not been reached, I add 2 to `X0` to get the next odd number and branch back to the start of the loop, finishing with a total accumulated sum of 2500 (`0x09C4`).

Task 6 – Sum Numbers Stored in Memory

Instruction: Write a loop to add all the numbers stored in memory. When finished, the result shall be stored in register x0.

Memory Initialization (provided)

```
// Set up base memory address
   MOVZ X7, #0x1000, LSL #16
// Store the numbers 1, 4, 1, 5, 9, 2 in memory
      MOVZ x7, #0x1000, LSL #16
      MOVZ x1, #1
      STUR x1, [x7, #0]
      MOVZ x1, #4
      STUR x1, [x7, #8]
      MOVZ x1, #1
      STUR x1, [x7, #16]
      MOVZ x1, #5
      STUR x1, [x7, #24]
      MOVZ x1, #9
      STUR x1, [x7, #32]
      MOVZ x1, #2
      STUR x1, [x7, #40]
```

Code

Listing 5: LEGv8 program to sum memory values

```
// Set up base memory address
MOVZ X7, #0x1000, LSL #16
```

```
// Store the numbers 1, 4, 1, 5, 9, 2 in memory
      MOVZ x7, #0x1000, LSL #16
      MOVZ x1, #1
      STUR x1, [x7, #0]
      MOVZ x1, #4
      STUR x1, [x7, #8]
      MOVZ x1, #1
      STUR x1, [x7, #16]
      MOVZ x1, #5
      STUR x1, [x7, #24]
      MOVZ x1, #9
      STUR x1, [x7, #32]
      MOVZ x1, #2
      STUR x1, [x7, #40]

      MOVZ x0, #0
      MOVZ x2, #0
      MOVZ x3, #48
loop:
      ADD x6, x7, x2
      LDUR x4, [x6, #0]
      ADD x0, x0, x4
      ADDI x2, x2, #8
      SUB x5, x3, x2
      CBNZ x5, loop

final:
```

Screenshots

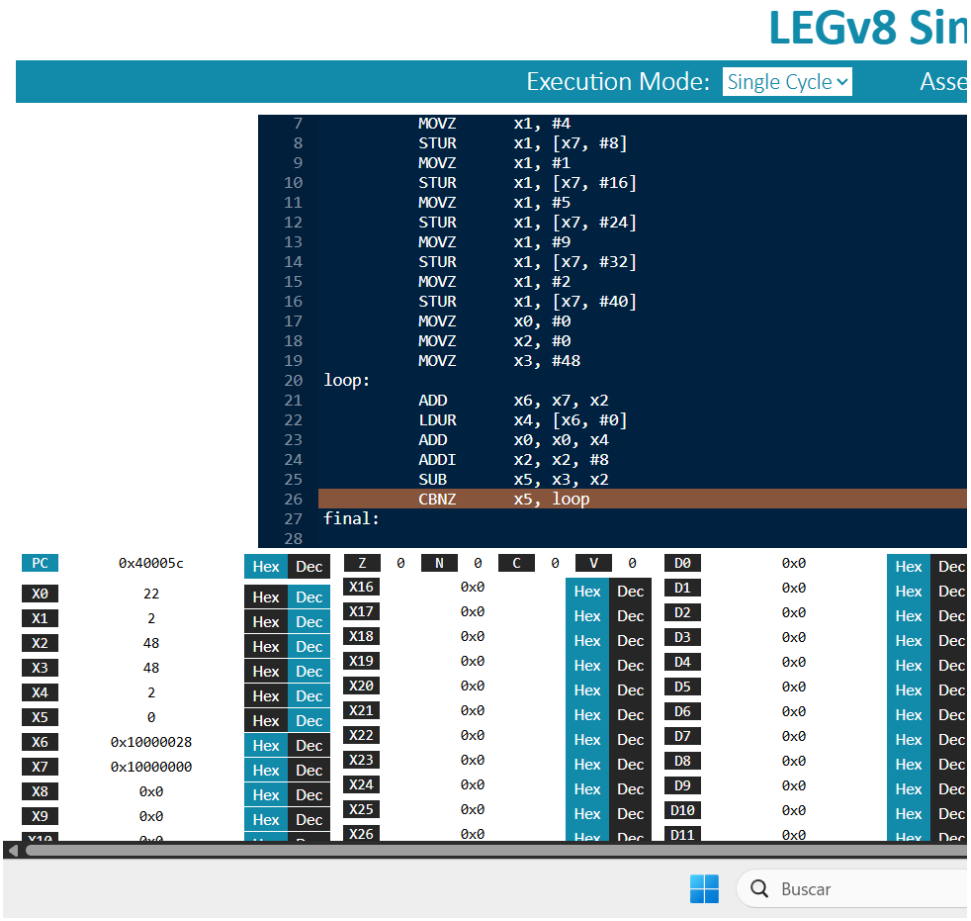


Figure 5: Execution of Task 6 in the LEGv8 simulator showing final values in registers.

Explanation

I use register `X7` as the base memory address (`0x10000000`), `X0` initialized to 0 as the accumulator for the result, and `X2` initialized to 0 as the byte offset. In every iteration, I calculate the effective memory address by adding the base address and offset into `X6`, and then load the value using `LDUR` into `X4`. Next, I add the loaded value to the partial result in `X0` and increment the offset in `X2` by 8 bytes (since each doubleword occupies 8 bytes). Finally, I subtract

`X2` from `48` into a temporary register (`X5`) to check if all 6 elements have been read; if the difference is not zero, the `CBNZ` instruction loops back to continue the process, finishing with a total accumulated sum of 22 in x0. (`0x16`).

Task 7 – The diagram below shows a simple combinational circuit

```
a. Make a truth table for the circuit.
b. Write a Boolean (logical) expression that describes the function.
c. Use ICs from the 7400 family to create the circuit physically.
```

Screenshots

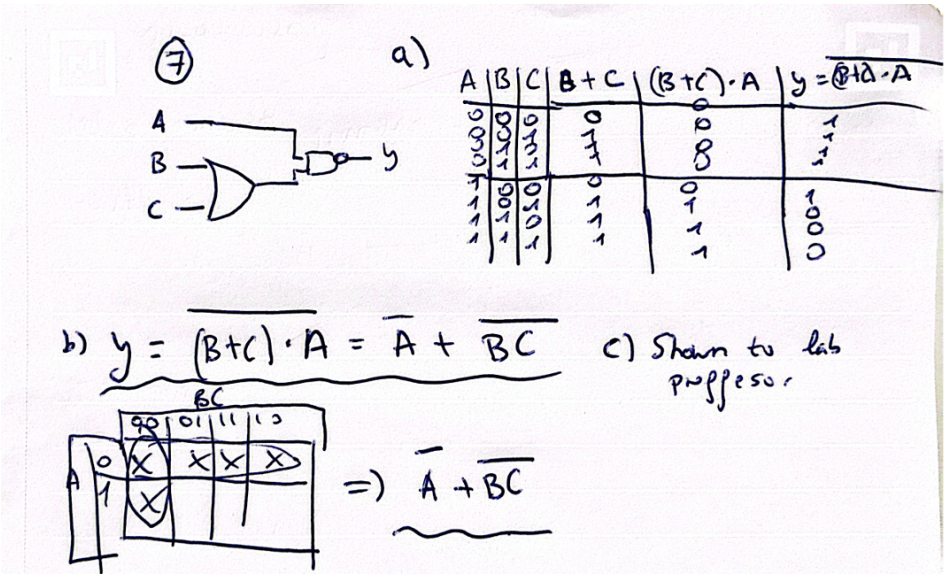


Figure 6: Execution of Task7 in the LEGv8 truth table,expression and logic circuit.

Task 8 – Boolean expression

A logic function is described by the Boolean expression:

$$F = !(A \cdot !B + C)$$

Screenshots

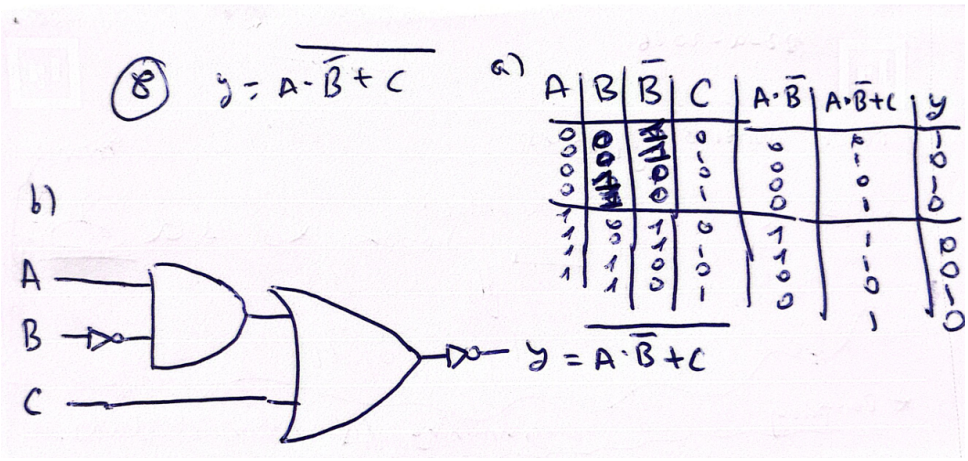


Figure 7: Execution of Task 8 showing truth table and logic circuit.

Task 9 – 2 bit adder

In this task, you will design a 2-bit adder by using digital gates. a. Make a truth table that describes a 2-bit adder. b. Draw a logic diagram (with logical gates) that implements the 2-bit adder. c. Implement the logic diagram in Logisim and show with a simulation that the circuit works properly.

Screenshots

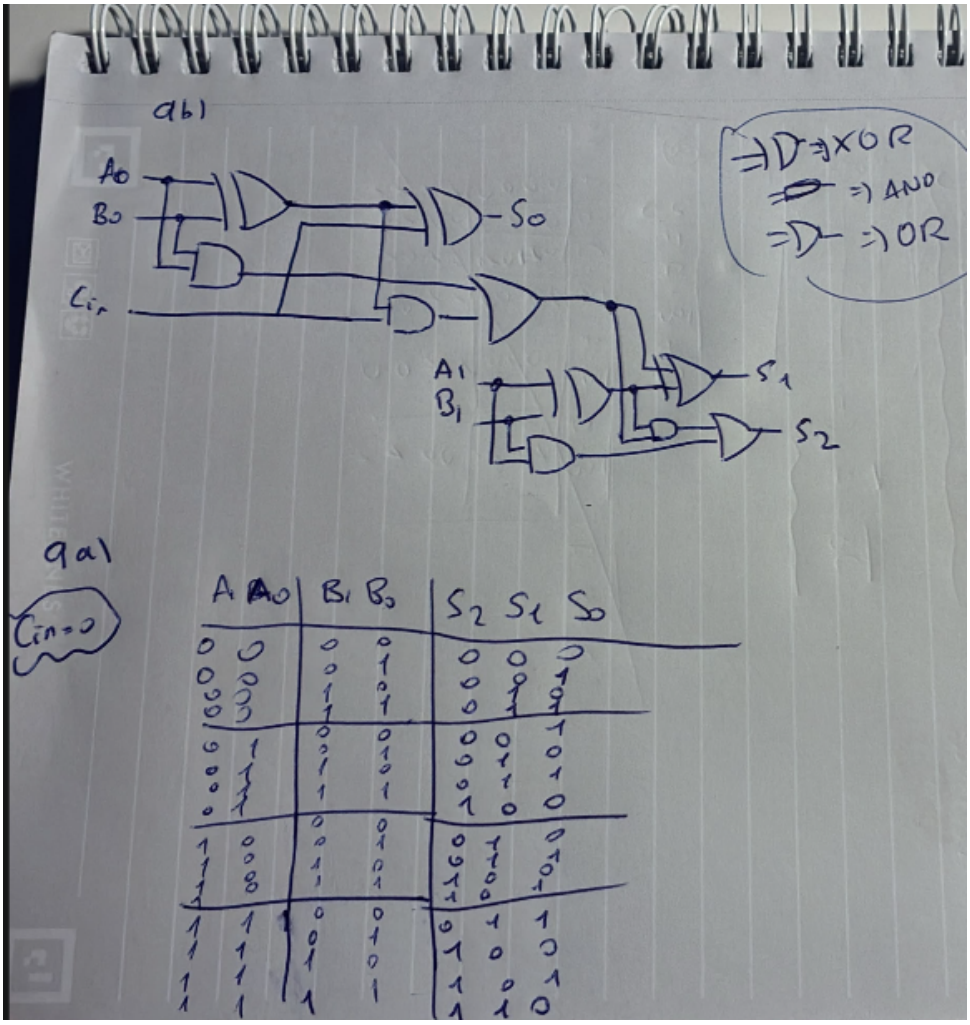


Figure 8: Execution of Task 9 in the LEGv8 simulator showing final values in registers.

Logism task

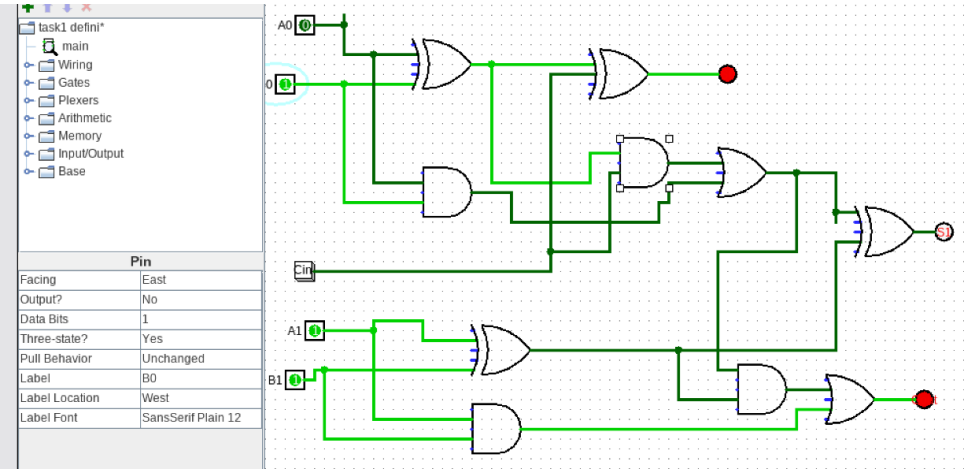


Figure 9: Execution of Task 9 showing 2 bit adder in Logism.

Task 10 – SR latch

Explanation

B)If our input to S is 1, Q will turn on and !Q will turn off, storing the value 1 (assuming R is 0)

C)If our input to S is 0, the SR maintains the previous state which was Q led turned on and !Q turned off

D)The moment you set the R to 1, it does not care the previous result, Q goes to 0 so Q led goes off and !Q led turns on. Both cannot, S and R cannot be 1, the behaviour is unclear

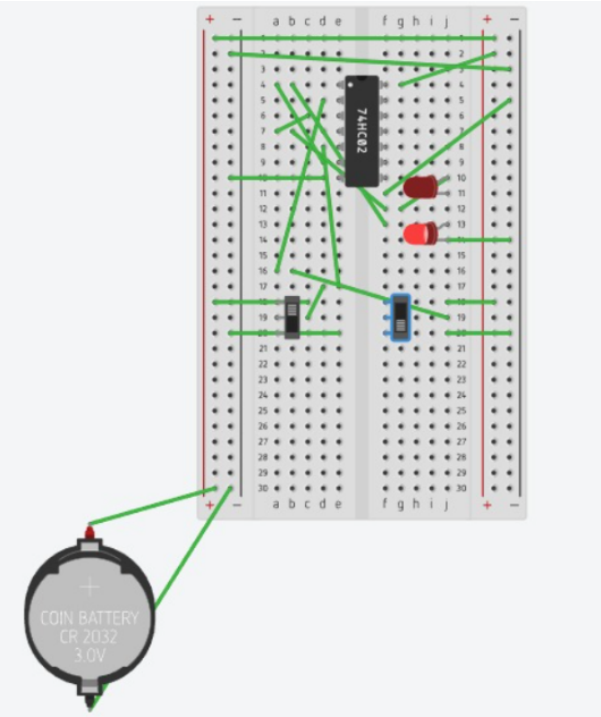


Figure 10: SR latch in online breadboard

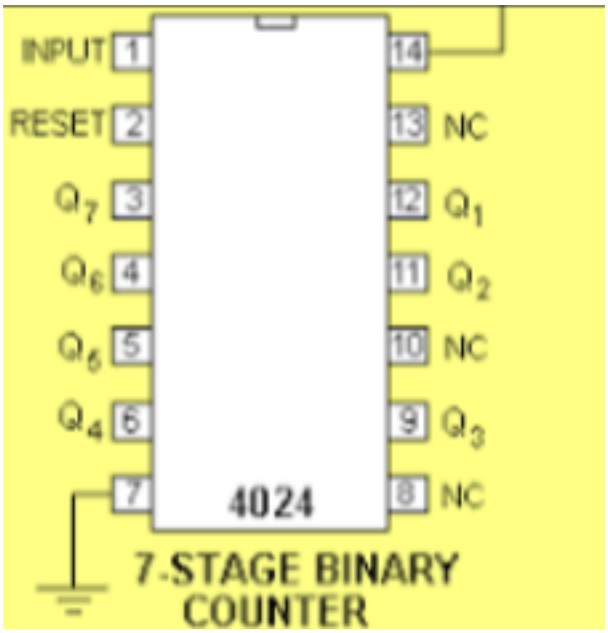


Figure 11: CMOS 4024 chip

Task 11 – Simple counter

Using a CMOS 4024 chip, which is the chip showed in the photo we build a simple 7 bit counter, we just connect the input to the clock in the red lab board, the reset to a switch and Q1 to led1, Q2 to led2, Q3 to led3, ... until Q7 is connected to led7, the we power and ground the chip and we have a counter that can be reseted and counts from 0 to 127.

*